

Análise e Desenvolvimento de Sistemas
Programação Orientada a Objetos
GPE01N30033

Trabalho Final (Grupo 6)

Integrantes:

Danilo Espíndola Folgieri Gomes – danilo.folgieri@a.ucb.br
Eduardo Sousa Daga – eduardo.daga@a.ucb.br
Fernando de Paula Hatabe – fernando.hatabe@a.ucb.br
Gabriel Sales do Nascimento – gabriel.snascimento@a.ucb.br

Professor: Alexandre Silva

1. Introdução:

Este documento explica como o nosso grupo evoluiu o Sistema de Biblioteca. Antes, o programa esquecia tudo quando era fechado. Agora, nós salvamos os dados de verdade usando um banco de dados chamado SQLite. Além disso, nós arrumamos a estrutura do projeto, organizando o código com os Design Patterns que aprendemos na última apresentação. Abaixo, explicamos de um jeito simples por que escolhemos cada padrão e como eles ajudam o nosso sistema a funcionar sem dar dor de cabeça.

2. Padrão Singleton (Conexão única):

- **Onde usamos:** Na classe `DatabaseConnection.java`
- **Por que usamos:** O banco de dados SQLite funciona criando um arquivo simples no computador (o `biblioteca.db`). Se várias partes do nosso código tentassem abrir e mexer nesse arquivo ao mesmo tempo, ele ia travar e dar erro.

O Singleton serve como um guarda de trânsito. Ele garante que o nosso programa inteiro use apenas uma única conexão aberta com o banco de dados. Se a gente não usasse, o sistema ficaria muito lento, porque ia ter que abrir e fechar o banco de dados toda vez que alguém quisesse salvar ou buscar um livro. Além disso, corríamos o risco de corromper o arquivo do banco.

3. Interfaces (Separando as tarefas):

- **Onde usamos:** Nas interfaces principais (`LivroRepository`, `UsuarioRepository` e `EmprestimoRepository`).
- **Por que usamos:** Usamos as interfaces para não misturar as coisas. A classe `Biblioteca` (que cuida das regras de emprestar e devolver) não faz a menor ideia de que os dados estão sendo salvos num SQLite. Ela simplesmente dá a ordem: "Salve esse usuário". Quem faz o trabalho pesado de ir lá no banco e salvar são as classes que implementam a interface (como a `UsuarioRepositorySQLite`).

A grande vantagem disso é que se amanhã alguém pedir para trocar o banco SQLite por um MySQL ou salvar tudo na nuvem, a gente não vai precisar mudar nenhuma regra de negócio na classe da biblioteca. É só criar um arquivo novo que segue a mesma interface.

4. Padrão Factory (Fábrica de usuários):

- **Onde usamos:** Na classe `UsuarioFactory.java`
- **Por que usamos:** No nosso menu (na classe `Main`), o usuário digita 1 para cadastrar um Aluno e 2 para cadastrar um Professor. Ficar fazendo um monte de if e else no meio do menu principal deixa o código feio e confuso.

A Factory resolveu isso. Ela é como uma fábrica mesmo. O menu só avisa para ela: "O número digitado foi 1", e ela constrói e devolve o objeto do Aluno prontinho, com o limite de 3 livros já configurado. Se o número for 2, ela devolve um Professor (com limite de 5 livros). Se um dia alguém quiser criar o usuário "Funcionário", a gente só mexe nessa fábrica, e o resto do programa continua funcionando igual.

5. Regras de Negócio:

Nosso sistema garante que tudo funcione direitinho usando o código e o banco de dados juntos:

- **Limite de empréstimos:** Um aluno só pode pegar 3 livros e um professor, 5. Antes de emprestar, o programa checka se a pessoa já atingiu o limite. Quando o empréstimo é feito, a gente aumenta o número de livros da pessoa lá no banco de dados.
- **Livro já emprestado:** O sistema sempre olha se o livro está marcado como "disponível". Se alguém já pegou, ele bloqueia.
- **Como funciona a devolução:** Quando alguém devolve um livro, o programa faz uma busca no banco de dados para ver com quem aquele livro estava. Depois, ele diminui a quantidade de livros na conta daquela pessoa, marca o livro como disponível de novo e apaga o registro do empréstimo.

6. Criação das Tabelas:

Para guardar tudo certinho, nós criamos três tabelas no banco de dados:

- `livros`: Guarda o número (ID), o título, o autor e um aviso se o livro está disponível ou não.
- `usuarios`: Guarda o nome da pessoa, se ela é Aluno ou Professor, qual o limite dela e quantos livros ela está segurando no momento.
- `emprestimos`: É uma tabela que faz a ponte. Ela só guarda o ID do livro e o ID da pessoa que pegou ele, unindo as duas coisas.